

把上述程序保存成 `test.sh` 后，再执行 `chmod +x test.sh`，即可用 `./test.sh` 来执行它。当然，扩展名也不是必需的，完全可以以不带扩展名的 `test` 命名。

上面的程序不是最简洁的（例如，可以直接把 `diff` 命令放在 `if` 语句中），但展示了 `bash` 脚本的一些其他用法。例如，`while` 循环是“`while <命令集>; do <命令集>; done`”，而 `if` 语句的基本是“`if <命令集>; do <命令集>;`”。不管是 `while` 还是 `if`，判断的都是命令集中最后一条语句的返回码（`exit code`）是否为 0。例如，若把上面的脚本改成 `if diff output.a output.b; then break; fi`，则当两个文件相同（`diff` 返回码为 0）时退出循环（这个不是我们所期望的）。如果忘记了命令格式，可以用 `help if` 和 `help while` 获取帮助。

上面的“`true`”和“`[`”都是程序。前者的作用是直接返回 0；而后者的作用是计算表达式（该程序要求最后一个参数必须是“`]`”），其中“`$?`”是 `bash` 内部变量，表示“上一个程序的返回码”。

A.2.3 再谈随机数

如果做过测试，可能会发现上面的方法有一个问题：如果程序执行太快，随机数生成器在相邻两次执行时，`time(NULL)` 函数返回值相同，因而产生出完全相同的输入文件。换句话说，每隔一秒才能产生出一个不同的随机数据。一个解决方案是利用系统自带的随机数发生器：在 Windows 下是环境变量 `%random%`，而在 `bash` 中是 `$RANDOM`。它们都是 0~32767 之间的随机整数。可以直接用脚本编写随机数生成器，也可以把它们传递到程序中。

A.3 编译器和调试器

既然编译器和调试器都是程序，执行方法和普通程序大致相同。在安装时，系统会自动把编译器和调试器程序所在路径加到搜索路径中，因此在执行时不必像 `./gcc` 这样加上路径名。

A.3.1 gcc 的安装和测试

尽管在现场比赛中，编译器都已安装好，但如果平时练习，一般需要自己安装。如果使用 Linux，在安装操作系统时即可选择安装 `gcc`、`g++`、`binutils` 等包，但若要在 Windows 中使用 C/C++ 语言，需要手工安装编译器。

本书推荐使用 MinGW 环境下的 `gcc`，它的好处是和 Linux 下的 `gcc` 一致性较好，而且是免费的。可以到 www.mingw.com 中下载最新的安装包，然后在安装时选择 `g++` 编译器。

安装完毕后，在命令行中执行 `gcc` 命令。如果显示 `gcc: no input files`，则安装成功；如果提示不存在这个命令，可能是因为没有把 `gcc` 所在目录加到搜索路径中。可以双击控制面板的“系统”图标，并在“高级”选项卡中设置环境变量。在“系统变量”中找到“`PATH`”（大小写无所谓），它就是可执行程序的搜索路径。请在它的最后加入 MinGW 安装路径的